

PaperLens AI

Visual Research Paper Explainer — Powered by LLMs

Next.js

Node.js

PostgreSQL

Python FastAPI

BullMQ + Redis

Docker Compose

LLM APIs

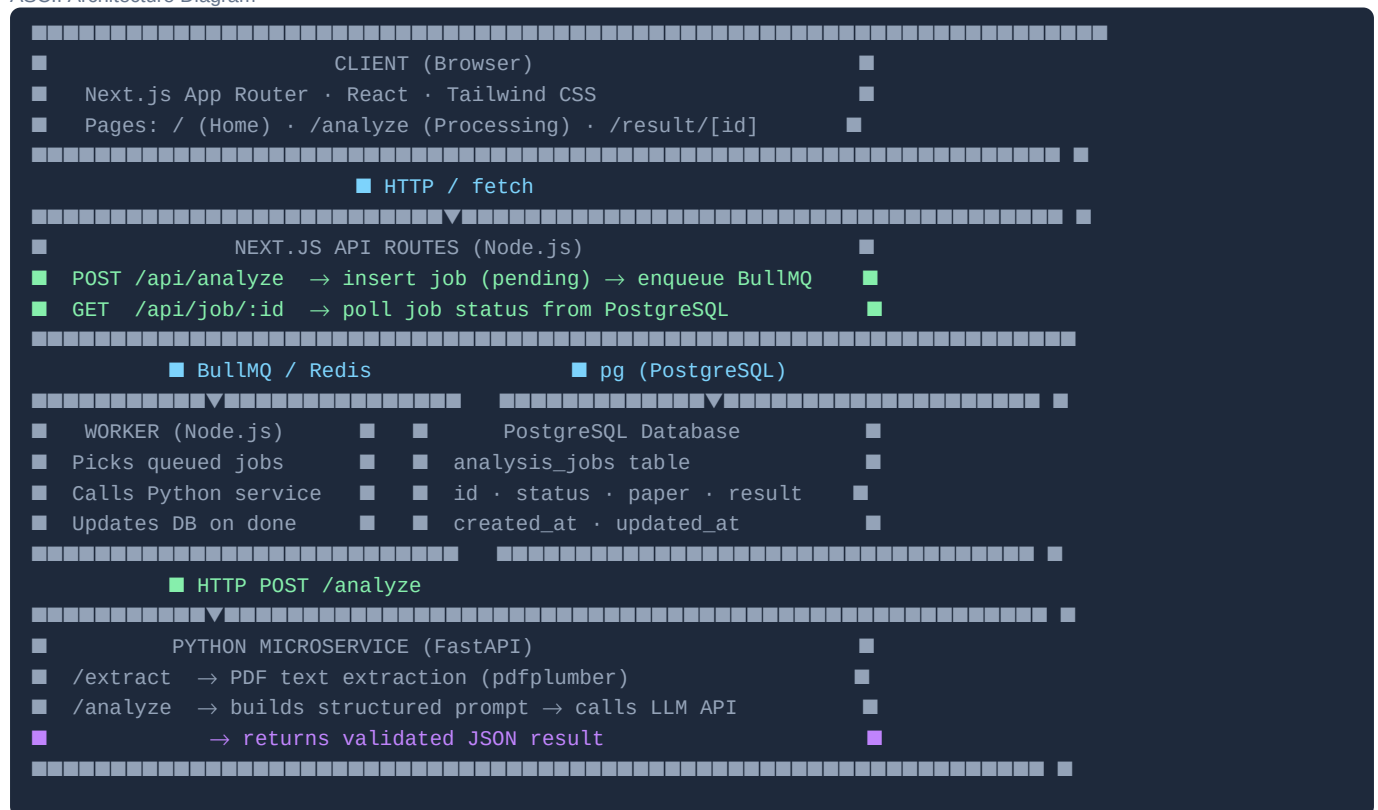
1. What PaperLens AI Does

PaperLens AI is a full-stack web application that transforms research papers into structured visual explanations. Users paste text, upload a PDF, or provide an arXiv/bioRxiv URL. After clicking **Analyze Paper**, the system processes the content asynchronously via an LLM and returns a polished result page containing: a **summary card**, **key concept chips**, **math breakdowns** with symbol tables, an **SVG mind map**, **learning flashcards**, and **related topic links** — all rendered from structured AI-generated JSON.

■ **Live URL:** <https://paperlens-ai.vercel.app> · **Repository:** <https://github.com/yourname/paperlens-ai>

2. Architecture Diagram & Layer Explanation

ASCII Architecture Diagram



Layer Explanations

Next.js Frontend

App Router pages, React components, polling loop, result rendering

Next.js API Routes

REST endpoints: create job, poll status, proxy to Python service

PostgreSQL

Persistent job store — status: pending → processing → completed → failed

BullMO + Redis

Async job queue — decouples HTTP request from slow LLM processing

Node.js Worker

Background process consuming BullMQ queue, calls Python, updates DB

Python FastAPI

PDF extraction, prompt construction, LLM API calls, JSON validation

LLM Provider

Gemini / Grog / OpenRouter — returns structured JSON output

3. Why This Stack & Architecture

Next.js — App Router

Chosen for SSR, built-in API routes (avoiding a separate Express server), file-based routing, and excellent DX. App Router enables React Server Components for faster initial page loads.

■ PostgreSQL

Relational structure is ideal for the job table with clear status transitions. ACID guarantees prevent duplicate processing. JSON columns store the AI result without needing a separate document store.

■ BullMQ + Redis

Provides durable, retryable background jobs. Decouples the HTTP response (instant job creation) from the slow LLM call (10–30s). Frontend polls for completion rather than holding an open connection, keeping the API stateless.

■ Python FastAPI Microservice

PDF extraction and AI prompt logic live in Python because the ecosystem (pdfplumber, PyMuPDF, LangChain) is richer than Node. Separating it as a microservice means the LLM provider can be swapped without touching Node code and the service can be scaled independently.

4. Full Folder Structure

```

paperlens-ai/
├── frontend/                                # Next.js application
│   ├── app/
│   │   ├── page.tsx                        # Home / input screen
│   │   ├── analyze/page.tsx               # Processing view (polling)
│   │   ├── result/[id]/page.tsx           # Visual result page
│   │   └── api/
│   │       ├── analyze/route.ts           # POST – create job
│   │       └── job/[id]/route.ts          # GET – poll status
│   ├── components/
│   │   ├── MindMap.tsx
│   │   ├── LearningCard.tsx
│   │   ├── MathSection.tsx
│   │   └── ProcessingView.tsx
│   └── lib/db.ts                          # pg pool singleton
├── worker/                                # Node.js BullMQ worker
│   ├── index.ts                           # queue consumer
│   └── processor.ts                       # calls Python, updates DB
├── python-service/                        # FastAPI microservice
│   ├── main.py                           # /extract + /analyze routes
│   ├── extractor.py                      # PDF → text (pdfplumber)
│   ├── prompt.py                         # prompt builder
│   └── requirements.txt
├── db/migrations/001_init.sql             # PostgreSQL schema
├── docker-compose.yml
├── .env.example
└── README.md

```

5. Step-by-Step Local Setup

Prerequisites

- Node.js 20+, npm / pnpm
- Python 3.11+, pip
- Docker + Docker Compose (recommended) OR PostgreSQL 15 + Redis 7 locally

Option A — Docker Compose (Recommended)

```

git clone https://github.com/yourname/paperlens-ai.git
cd paperlens-ai
cp .env.example .env          # fill in your LLM API key
docker compose up --build     # starts all 5 services
# App available at http://localhost:3000

```

Option B — Manual Setup

1. PostgreSQL

```

createdb paperlens
psql paperlens < db/migrations/001_init.sql

```

2. Python Microservice

```

cd python-service
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --port 8000 --reload

```

3. Next.js Frontend

```
cd frontend
npm install
npm run dev      # http://localhost:3000
```

4. Node.js Worker (separate terminal)

```
cd worker
npm install
npm run dev
```

6. .env.example Walkthrough

```
# Database
DATABASE_URL=postgresql://postgres:password@localhost:5432/paperlens

# Redis (BullMQ)
REDIS_URL=redis://localhost:6379

# Python microservice URL (called by Node worker)
PYTHON_SERVICE_URL=http://localhost:8000

# LLM Provider — uncomment one
GEMINI_API_KEY=your_gemini_key_here
# GROQ_API_KEY=your_groq_key_here
# OPENROUTER_API_KEY=your_openrouter_key_here

# Next.js
NEXTAUTH_SECRET=your_random_secret_here
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

7. All API Endpoints

POST /api/analyze — Create analysis job

```
# Request
POST /api/analyze
Content-Type: application/json
{
  "title": "Attention Is All You Need",
  "content": "We propose a new network architecture...",
  "inputType": "text"      // "text" | "pdf" | "url"
}

# Response 202 Accepted
{
  "jobId": "clx9f3k2b0000abc123",
  "status": "pending"
}
```

GET /api/job/:id — Poll job status

```
# Response — still processing
{ "status": "processing", "step": "Finding key ideas" }

# Response — completed
{
  "status": "completed",
  "result": {
    "summary": { "title": "...", "category": "...", "difficulty": "...", ... },
    "concepts": ["Transformers", "Attention", "..."],
    "math": { "hasEquation": true, "equation": "...", "symbols": [], ... },
    "mindMap": { "center": "...", "branches": [] },
    "learningCards": [ { "question": "...", "answer": "..." } ],
    "relatedTopics": [ "Topic1", "Topic2" ]
  }
}

# Response — failed
{ "status": "failed", "error": "LLM timeout" }
```

POST /analyze — Python microservice

```
# Request
POST http://localhost:8000/analyze
{ "text": "full paper text here..." }

# Response
{
  "summary": { "title": "...", "oneLiner": "...", "problem": "...", "method": "..." },
  "concepts": ["Concept1", "Concept2"],
  "math": { "hasEquation": true, "equation": "Attention(Q,K,V) = ...", ... },
  "mindMap": { "center": "...", "branches": [...] },
  "learningCards": [ ... ],
  "relatedTopics": [ ... ]
}
```

8. Exact AI Prompt Used

Design principle: The system prompt instructs the model to return pure JSON only — no markdown fences, no preamble — so the response can be `JSON.parse()`d directly without any post-processing or regex stripping.

SYSTEM:

You are PaperLens AI. When given a research paper, return ONLY a JSON object with this structure (no markdown, no backticks, no preamble):

```
{
  "summary": {
    "title": "...",
    "category": "...",
    "difficulty": "beginner | intermediate | advanced | expert",
    "oneLiner": "...",
    "problem": "...",
    "method": "..."
  },
  "concepts": ["Concept1", "Concept2", ...],
  "math": {
    "hasEquation": true,
    "equation": "Attention(Q,K,V) = softmax(QKT / sqrt(d_k)) * V",
    "meaning": "...",
    "symbols": [{ "symbol": "Q", "meaning": "Query matrix" }],
    "humanExplanation": "Plain English 2-3 sentence explanation..."
  },
  "mindMap": {
    "center": "Core concept (2-3 words)",
    "branches": [{ "topic": "...", "detail": "...", "leaves": ["..."] }]
  },
  "learningCards": [{ "question": "...", "answer": "..." }],
  "relatedTopics": ["Topic1", "Topic2", ...]
}
```

If no math equation found, set hasEquation to false.

Always provide all sections. Make explanations beginner-friendly.

USER:

Analyze this research paper and return the JSON:

<paper text inserted here>

9. How Async Processing Works

The analysis pipeline is deliberately asynchronous to avoid HTTP timeouts from slow LLM calls (10–30s).

- **Step 1 — Request:** Frontend POSTs to `/api/analyze`. Node inserts a DB row with `status='pending'`, enqueues a BullMQ job with the `jobId`, and immediately returns 202 with the `jobId`.
- **Step 2 — Queue:** BullMQ stores the job in Redis. The worker process (running separately) consumes jobs with `concurrency=2`.
- **Step 3 — Processing:** Worker updates DB status to `'processing'`, calls Python `/analyze` with paper text, waits for LLM response.
- **Step 4 — Completion:** Worker stores JSON result in `result_json` column, updates status to `'completed'` (or `'failed'` on error).
- **Step 5 — Retry:** BullMQ retries failed jobs up to 3 times with exponential backoff before marking permanently failed.

10. How Frontend Detects Completion

```
// app/analyze/page.tsx (simplified)
const [status, setStatus] = useState('pending')

useEffect(() => {
  const interval = setInterval(async () => {
    const res = await fetch(`/api/job/${jobId}`)
    const data = await res.json()

    if (data.status === 'completed') {
      clearInterval(interval)
      router.push(`/result/${jobId}`) // navigate to result page
    }
    if (data.status === 'failed') {
      clearInterval(interval)
      setError(data.error) // show retry UI
    }
  }, 2000) // polls every 2 seconds

  return () => clearInterval(interval)
}, [jobId])
```

The processing view animates through step labels locally using a timer while polling. When the DB row flips to `'completed'`, the router pushes to the result page which reads the full JSON and renders all UI sections.

11. Database Schema

```
-- db/migrations/001_init.sql
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS analysis_jobs (
  id          TEXT PRIMARY KEY DEFAULT gen_random_uuid()::text,
  status      TEXT NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'processing', 'completed', 'failed')),
  input_type  TEXT DEFAULT 'text',
  paper_title TEXT,
  paper_text  TEXT,
  result_json JSONB,
  error       TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_jobs_status      ON analysis_jobs(status);
CREATE INDEX IF NOT EXISTS idx_jobs_created_at ON analysis_jobs(created_at DESC);
```

12. Known Bugs & Limitations

- PDF extraction degrades on scanned / image-only PDFs (no OCR in v1)
-

- arXiv URL fetching extracts abstract only — not full PDF text
- Mind map SVG can overflow with very long branch or leaf names
- LLM occasionally returns malformed JSON on very short inputs — needs retry logic
- No authentication — jobs accessible by jobId (UUID) only
- Worker and Next.js share DB connection pool — should be separated at scale

13. What I Would Improve With More Time

- **OCR pipeline** — Integrate Tesseract for scanned PDFs to extract text from images
- **Chat with paper** — RAG-based Q&A; using pgvector embeddings (pgvector extension)
- **Full arXiv fetch** — Download and parse the full PDF from arXiv abstract URLs
- **React Flow mind map** — Replace SVG with interactive, collapsible React Flow nodes
- **Export to PDF** — Let users download the visual summary as a shareable PDF
- **Paper history** — Search, tag, and compare past analyses
- **WebSocket updates** — Replace polling with real-time job progress via WebSockets
- **Auth + accounts** — NextAuth.js with Google OAuth so users own their paper library
- **Quiz generation** — Auto-generate comprehension questions from learning cards

14. Deployment

Frontend (Next.js)	Vercel — auto-deploy on push to main branch
PostgreSQL	Neon (serverless Postgres) or Railway
Redis	Upstash Redis (serverless, generous free tier)
Python Microservice	Railway or Render (Docker container)
Worker (Node.js)	Railway background worker service

Production Environment Variables

```
DATABASE_URL=postgresql://...neon.tech/paperlens?sslmode=require
REDIS_URL=rediss://...upstash.io:6380
PYTHON_SERVICE_URL=https://paperlens-python.railway.app
GEMINI_API_KEY=AIZA...
NEXTAUTH_SECRET=...
NEXT_PUBLIC_APP_URL=https://paperlens-ai.vercel.app
```

Note: Replace all placeholder values (yourname, API keys, live URLs, college name) before submitting. This README was generated as part of the PaperLens AI engineering assignment.